

How Far Off the Recorded Path Can a Gaussian-Splat Driving Simulator Be Trusted?

CS 566 Project Proposal · Kyle Sha (ID 9085525476) · Allen Xia (ID 9085517234) · 29 Sep 2026

Main papers: [1] **Difix3D+** (CVPR 2025 Oral), *Difix training re-implemented (progressive 3D update as an extension)* · [2] **FaithFusion** (CVPR 2026), *compared* · [3] **HUGSIM** (TPAMI 2026), *reference splat-based simulator*

1. Problem Statement

Driving software is increasingly tested in simulators rebuilt from real driving logs. The recorded video becomes a **3D Gaussian splat** [13], a scene of millions of small, semi-transparent colored ellipsoids that can be rendered from any camera position, so the simulated vehicle can leave the recorded trajectory. The reconstruction is only reliable near that trajectory, however. After a lane change, or with a higher camera (e.g., a truck), renders degrade, and a driving system tested on them may behave differently than on the road. We want to find out **how far from the recorded trajectory a splat-based simulator can still be trusted, and whether current generative repair methods ("fixers") restore that trust or only make the images look more realistic**. Here, **trust** means the agreement between the outputs of a lane detector and a vehicle detector on a rendered image and on the **real** image captured at the same pose.

2. Significance and Motivation

Simulation is one of the main ways to validate driving software at scale. With unreliable off-trajectory renders, a simulator may accept unsafe behavior or reject safe behavior. Both authors are members of Wisconsin Autonomous, a student autonomous-vehicle team, where we develop and test driving software in conventional simulators (MATLAB/Simulink, RoadRunner, CARLA). Their hand-built 3D assets look visibly different from real camera images, which limits how well results carry over to the vehicle. Reconstruction-based simulators could close this gap, but only if renders stay faithful off the recorded trajectory. A generative fixer can make an image more convincing yet less accurate, and this trade-off motivates the project.

3. State of the Art

HUGSIM [3] runs driving policies inside reconstructed scenes, and the EUVS benchmark [7] shows that current methods degrade under large viewpoint changes.

Difix3D+ [1] repairs off-trajectory renders with a single-step diffusion model and adds the repaired views back as training images for the 3D scene. It reports roughly half the FID (a realism score; lower is better) of prior methods. NVIDIA Fixer [4] applies the same approach to driving data. FaithFusion [2] restricts which pixels a fixer may overwrite, using a per-pixel map of how well the reconstruction already supports each pixel.

These fixers are mostly evaluated with image-realism metrics (FID, LPIPS), or with shifted cameras for which no real image exists (PRG-Fusion [6]). ReconDreamer [5] scores shifted-view renders with lane and vehicle detectors, but against projected annotations rather than real images at the new pose. The MV2 dataset [9], Concept to Capability [10] and SetDiff [11] use offset analysis or real off-trajectory images, but mainly with image-similarity metrics. We found few comparisons of fixers against real off-trajectory images that also count invented content.

4. Proposed Approach

We organize the work in three tiers. The **core** study uses only public data, released code and weights, and standard training, and is a complete project on its own. **Extensions** and **stretch goals** are added in order as time permits; each adds methods or test conditions to the same evaluation.

Core: fixers against real off-trajectory images. Para-Lane [8] scans the same roads in three parallel lanes (approximately 3 m and 6–7.7 m apart). We reconstruct a splat from one lane with gsplat [13] and render it at the neighboring lanes' recorded poses, where real images exist. We compare the raw render, the released Difix [1] and NVIDIA Fixer [4], and our **re-implementation of the Difix training procedure**. As in the paper, our fixer is fine-tuned from the pretrained SD-Turbo diffusion model. We write the training code from the paper and validate it against NVIDIA's official code and weights. It is trained on driving data, with pairs built from neighboring lanes, each consisting of a render moved into lane B and the real lane-B image (cf. ReCamDriving [12]). All methods are scored for

image similarity, lane and vehicle agreement, and **hallucination** (invented or erased lane markings and spurious vehicles).

Extensions are (a) a **smaller fixer**, a standard 15M-parameter U-Net trained from scratch on the same pairs, to see how much of Difix's behavior it keeps; (b) the **road-surface prior** from ViSE [15], which fits a smooth road surface to LiDAR and involves no image generation (we re-implement it because no code is released); (c) the **progressive 3D update** of Difix3D+ [1], which adds repaired views back as training images for the 3D scene; and (d) **more test roads** and the **heading axis**.

Stretch goals are (e) **camera height**, using MV2 [9], in which a car and a scooter record the same road at different camera heights; (f) a **cross-dataset evaluation** on US city scenes (EUVS benchmark [7], built on the MARS dataset) and Indian roads (MV2 [9]); (g) **FaithFusion** [2] as an additional comparison method; and (h) **closed-loop evaluation** in a splat-based driving simulator (e.g., HUGSIM [3]).

5. Open Questions and Expected Findings

Fixers are diffusion models trained for realism, so they may add plausible detail where the true scene is unknown. Realism metrics alone would not show this. Without a real image at the new pose, an evaluation cannot tell a faithful repair from a convincing invention; neighboring-lane captures provide that image, both for scoring and as training targets. We therefore ask:

- How quickly does agreement with the real image fall as the camera moves sideways (core), turns (extension) or changes height (stretch)?
- Do fixers improve lane and vehicle agreement, or mainly image realism? Do they introduce spurious lane markings or vehicles?
- Does training a fixer on real neighboring-lane pairs make it more faithful (core)? How much of the effect remains with a much smaller fixer or a purely geometric road prior (extensions)?
- Do fixers trained on one dataset help on another (stretch)?

We expect the answers to depend on the scene and the type of deviation. For example, fixers may improve realism more than lane accuracy, and a fixer fine-tuned on one city may transfer only partly to others.

6. Evaluation Plan and Expected Results

We test three deviation axes. Lateral offset (core) uses real images from the adjacent lanes at ≈ 3 m and 6–7.7 m. Heading is an extension; for a pure rotation the real image can be warped exactly to the new view, so only pixels the original camera saw are scored. Camera height (stretch) uses the real car and scooter views from MV2 or, as a fallback, the real image re-projected onto the road plane at the new height.

We measure image similarity to the real image with PSNR and LPIPS (standard pixel-level and perceptual similarity scores). **Lane-marking precision and recall** compare painted-marking pixels (white and yellow paint found by a color and local-contrast threshold in the road region) and give the fraction of marking pixels in the render that match real markings and the fraction of real markings reproduced, i.e. invented versus erased markings. **Lane agreement** is the F1 score between lanes detected by CLNet [14] on the render and on the real image; vehicle agreement is defined the same way with YOLO11, following ReconDreamer [5]. **Spurious vehicles** are vehicles detected in the render with no counterpart in the real image. As a stretch goal, closed-loop evaluation lets a released driving policy drive inside a splat-based simulator and scores it on safety and progress.

Expected results. For the core we plan to show trust versus lateral offset (Figure 1), hallucination statistics for each method (Table 1), our Difix re-implementation compared with NVIDIA's official code (Table 2), and side-by-side videos with hallucinated content annotated. Extensions add methods to Figure 1 and Table 1, plus a heading curve; stretch goals add trust versus camera height (Figure 2), cross-dataset results (Table 3) and closed-loop scores.

Components and how each is used:

Component (what it does)	Source	Our use
Difix image fixer [1]: a diffusion model that takes a flawed rendered image and outputs a cleaned-up version	NVIDIA (code, weights)	Re-implemented (our training code) and trained on our pairs
Progressive 3D update [1]: cleaned images are added back as training views, so the 3D scene itself improves off the recorded path	Difix3D+ paper	Re-implemented

Component (what it does)	Source	Our use
FaithFusion [2]: limits which pixels a fixer may change, using a per-pixel confidence map	Released code	Comparison method (run as released)
NVIDIA Fixer [4]: NVIDIA's version of Difix, trained on its own driving data	Released weights	Comparison method (run as released)
Smaller fixer : standard image-to-image U-Net (15M parameters)	Standard architecture	Trained by us from scratch
Road-surface prior [15]: fits a smooth road surface to LiDAR points and flattens the road in the 3D scene; involves no image generation	ViSE paper (no code)	Re-implemented
3D reconstruction [13]: builds each Gaussian-splat scene from the recorded video	gsplat library	Run by us per scene
Detectors and simulator : CLNet lanes [14], YOLO11 vehicles; for closed-loop evaluation, a splat-based simulator (e.g., HUGSIM [3]) with a released driving policy	Released, pretrained	Used as-is (evaluation only)

7. Timeline

Dates	Tasks
9/29–10/12	Core : splats of the Para-Lane test roads; evaluation framework (detectors, metrics); raw, released Difix and NVIDIA Fixer results
10/13–10/29	Core : Difix training re-implementation, validation against the official code, training on neighboring-lane pairs; core figures and tables; mid-term report (10/29)
10/30–11/20	Extensions : smaller fixer, road-surface prior, progressive update, more roads, heading; stretch goals as time permits (camera height, cross-dataset, FaithFusion, closed loop)
11/21–12/10	Final figures and videos; presentation (12/1–12/10); project webpage (12/10)

Risks. The core needs only public data, released code and weights, and standard fine-tuning, and is scheduled to be complete by the mid-term report. The project therefore yields a full result even if extensions or stretch goals are not reached. Results may rest on few test roads, so we will use as many scenes as time permits and report per-scene variance. Closed-loop evaluation is limited to simulators and driving policies that are released and run on current software.

References

1. Difix3D+, CVPR 2025 · arxiv.org/abs/2503.01774 · github.com/nv-tlabs/Difix3D
2. FaithFusion, CVPR 2026 · arxiv.org/abs/2511.21113 · github.com/wangyuanbiubiubiu/FaithFusion
3. HUGSIM, TPAMI 2026 · arxiv.org/abs/2412.01718 · github.com/hyzhou404/HUGSIM
4. NVIDIA Fixer, 2025 · github.com/nv-tlabs/Fixer
5. ReconDreamer, CVPR 2025 · arxiv.org/abs/2411.19548
6. PRG-Fusion, 2026 · arxiv.org/abs/2609.06948
7. EUVS benchmark, ICCV 2025 · arxiv.org/abs/2412.05256
8. Para-Lane dataset, 3DV 2025 · arxiv.org/abs/2502.15635
9. MV2 dataset, ECCV 2026 · arxiv.org/abs/2608.12442
10. Concept to Capability, SafeComp 2026 · arxiv.org/abs/2605.01995
11. SetDiff, 2026 · arxiv.org/abs/2601.07540
12. ReCamDriving, ECCV 2026 · arxiv.org/abs/2512.03621
13. 3D Gaussian Splatting, SIGGRAPH 2023 · arxiv.org/abs/2308.04079
14. CLNet, CVPR 2022 · arxiv.org/abs/2203.10350
15. ViSE, ICCV 2025 · arxiv.org/abs/2510.18341